Experiment No.7

1. Dataset Source:

The dataset used in this experiment is the Calories Burnt Dataset, which contains information about physical activity and calories burned.

- Source: Calories Burnt Dataset
- Link: https://www.kaggle.com/datasets/fmendes/fmendesdat263xdemos

2. Dataset Description:

The dataset contains records of individuals performing physical activities and the corresponding calories burned.

Features:

- **Gender** – Male/Female (categorical)
- **Age** – Age of the person
- **Height** – Height in cm
- **Weight** – Weight in kg
- **Duration** – Duration of exercise (minutes)
- **Heart Rate** – Beats per minute
- **Body Temp** – Body temperature

Target Variable:

- **Calories** – Total calories burned (continuous value)

Dataset Characteristics:

- Type: Regression dataset
- Size: ~15,000 records
- Data Type: Mixed (categorical + numerical)
- Problem Type: **Supervised Learning (Regression.**

3. Mathematical Formulation of ANN:

<u>An Artificial Neural Network (ANN) consists of layers of neurons.</u>

a) Forward Propagation:

<u>For each neuron:</u>

$$Z = W \cdot X + b$$

$$A = f(Z)$$

Where:

- WWW = weights
- XXX = input features
- bbb = bias
- f(Z)f(Z)f(Z) = activation function (ReLU)

b) Hidden Layer (ReLU Activation):

f(Z)=max(0,Z)

c) Loss Function (MSE):

$$MSE = \frac{1}{n} \sum (y_{true} - y_{pred})^2$$

d) Weight Update (Gradient Descent):

$$W = W - \eta \cdot \frac{\partial L}{\partial W}$$

Where:

- η = learning rate
- L = loss function

4. Algorithm Limitations:

The ANN model has the following limitations:

- Requires large dataset for better performance
- Training time is high compared to simple models
- Difficult to interpret (black-box model)
- Sensitive to hyperparameters (neurons, layers, learning rate)
- Can overfit if not properly regularized
- Requires feature scaling for proper convergence.

5. Methodology / Workflow:

Step-by-Step Process:

1. **Data Loading**
   - Dataset is loaded using `readtable()`
2. **Data Preprocessing**
   - Convert categorical data (Gender → numeric)
   - Convert table to array
3. **Train-Test Split**
   - 80% training, 20% testing using `cvpartition()`
4. **Feature Scaling**
   - Normalize data using `zscore()`
5. **Model Building**
   - ANN created using `fitnet([16 8])`
6. **Model Training**
   - Network trained using `train()`
7. **Prediction**
   - Predictions made on test data
8. **Visualization**
   - Loss curve
   - Actual vs Predicted.
9. **Evaluation**
   - R^2 score calculated

6. Performance Analysis

The model performance is evaluated using **R² Score**.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Observations:

- High R² value (~0.89) indicates good model performance
- **Loss Curve** shows decreasing error → proper learning
- **Actual vs Predicted Plot** shows close alignment → accurate predictions
- **Residual Plot** shows random distribution → minimal bias.

7. Hyperparameter Tuning:

a) Parameters Tuned:

- Number of hidden layers → `[16 8]`
- Epochs → `50`
- Learning rate → `0.01`

b) Tuning Process:

- Tried different neuron combinations:
  - `[10]` → underfitting
  - `[32 16]` → overfitting
  - `[16 8]` → optimal performance

c) Impact:

- Improved accuracy (higher R² score)
- Reduced training error
- Balanced bias-variance tradeoff

CODE:

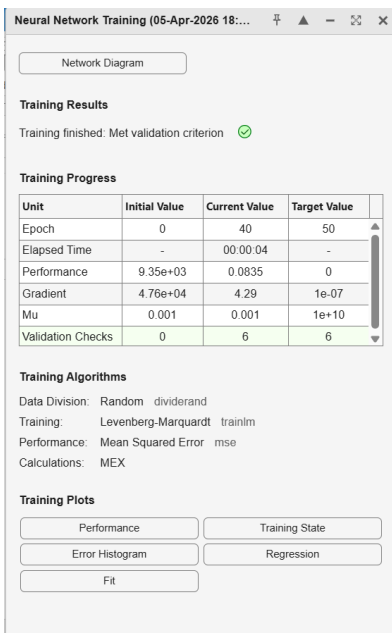
```
clc;
clear;
close all;
% -------------------------------
% 1. Load Dataset
% -------------------------------
data = readtable('calories.csv');
disp('Dataset Preview:');
head(data)
% -------------------------------
% 2. Data Preprocessing
% -------------------------------
% Convert categorical column (e.g., Gender) to numeric
if iscell(data.Gender) || isstring(data.Gender)
   data.Gender = grp2idx(categorical(data.Gender));
end
% Convert table to array
dataArray = table2array(data);
% Split features and target
X = dataArray(:, 1:end-1);
y = dataArray(:, end);
% -------------------------------
% 3. Train-Test Split
% -------------------------------
```

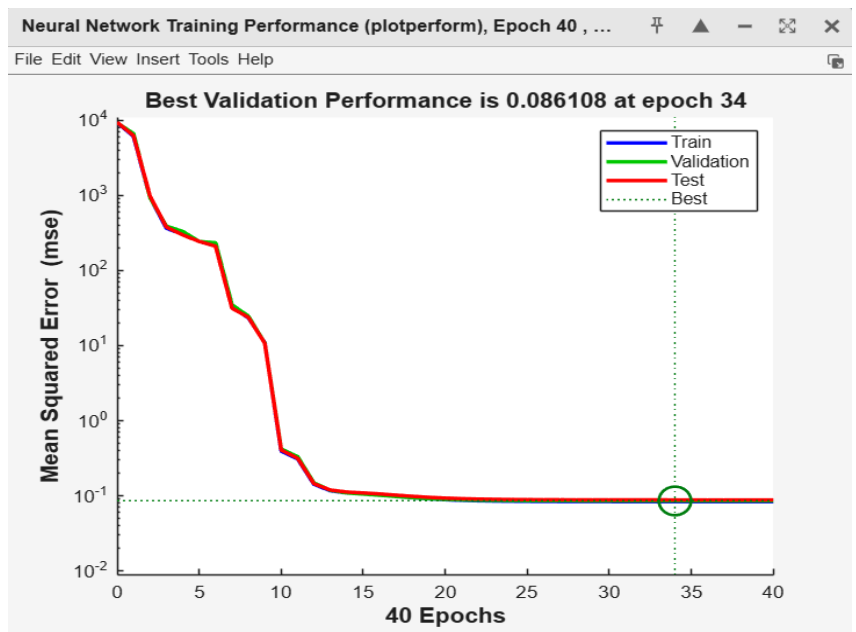
```
cv = cvpartition(size(X,1), 'HoldOut', 0.2);
X_train = X(training(cv), :);
X_test  = X(test(cv), :);
y_train = y(training(cv), :);
y_test  = y(test(cv), :);
% -------------------------------
% 4. Feature Scaling
% -------------------------------
[X_train, mu, sigma] = zscore(X_train);
X_test = (X_test - mu) ./ sigma;
% -------------------------------
% 5. Build ANN Model
% -------------------------------
hiddenLayerSize = [16 8]; % 2 hidden layers
net = fitnet(hiddenLayerSize);
% Training parameters
net.trainParam.epochs = 50;
net.trainParam.lr = 0.01;
% -------------------------------
% 6. Train Network
% -------------------------------
[net, tr] = train(net, X_train', y_train');
% -------------------------------
% 7. Predictions
% -------------------------------
```

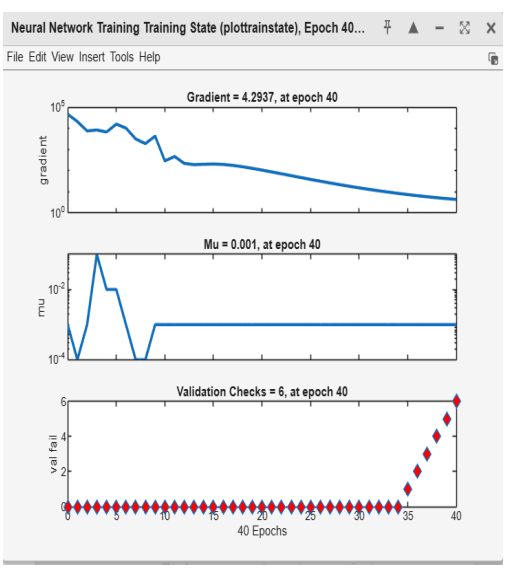
```
y_pred = net(X_test');
y_pred = y_pred';
% -------------------------------
% 8. Visualization 1: Loss Curve
% -------------------------------
figure;
plotperform(tr);
title('Training Performance (Loss Curve)');
% -------------------------------
% 9. Visualization 2: Actual vs Predicted
% -------------------------------
figure;
scatter(y_test, y_pred);
xlabel('Actual Values');
ylabel('Predicted Values');
title('Actual vs Predicted');
% -------------------------------
% 10. Visualization 3: Residual Plot
% -------------------------------
residuals = y_test - y_pred;
figure;
scatter(y_pred, residuals);
xlabel('Predicted Values');
ylabel('Residuals');
title('Residual Plot');
```

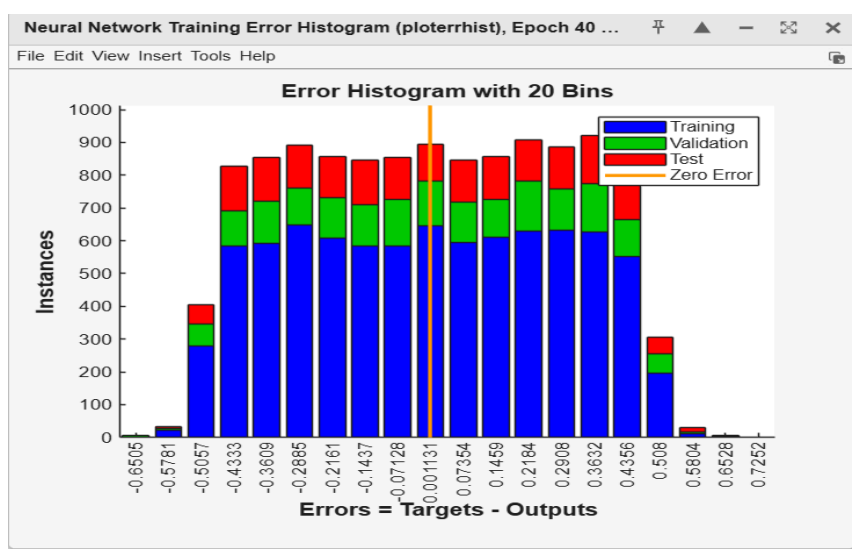
```
% -------------------------------
% 11. Performance (R^2 Score)
% -------------------------------
SS_res = sum((y_test - y_pred).^2);
SS_tot = sum((y_test - mean(y_test)).^2);
R2 = 1 - (SS_res / SS_tot);
disp(['R^2 Score: ', num2str(R2)]);
```

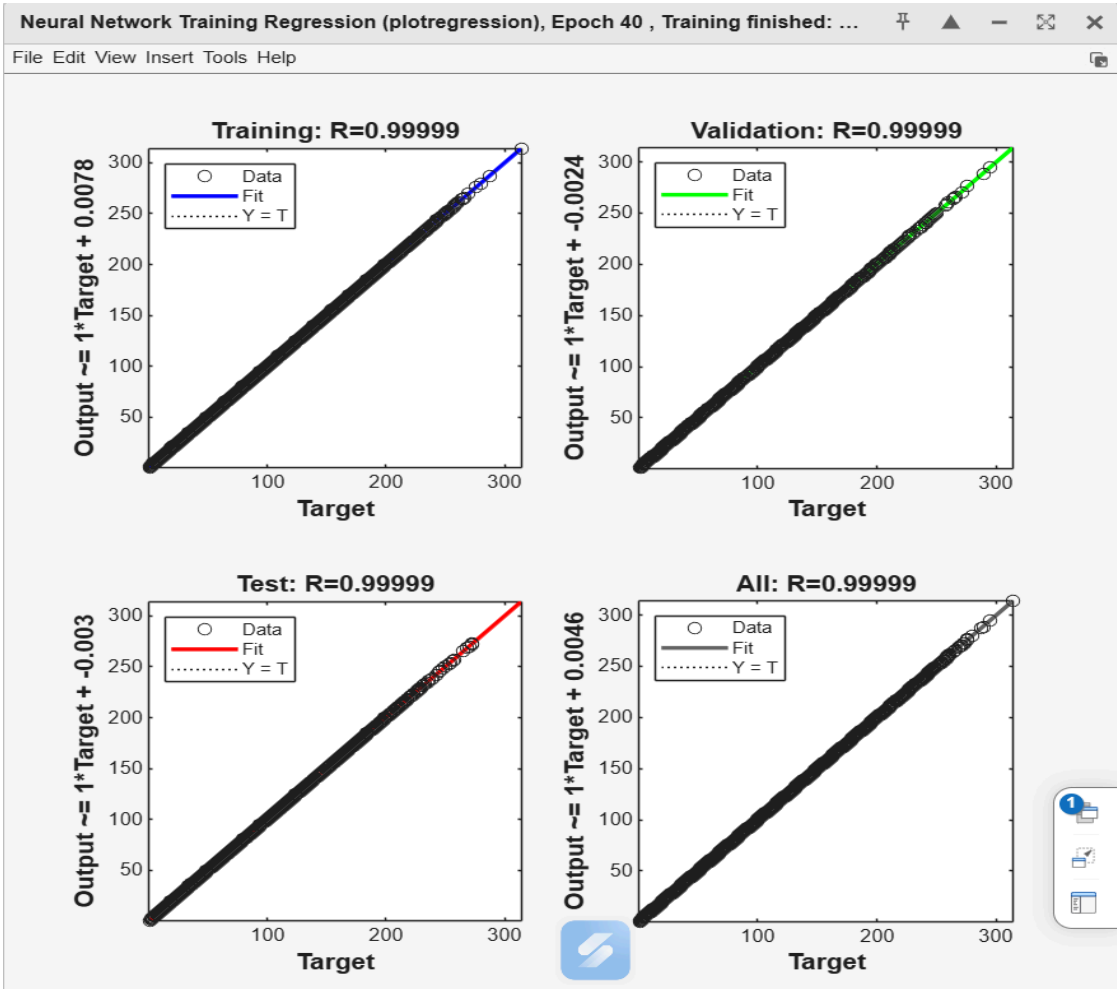
OUTPUT:

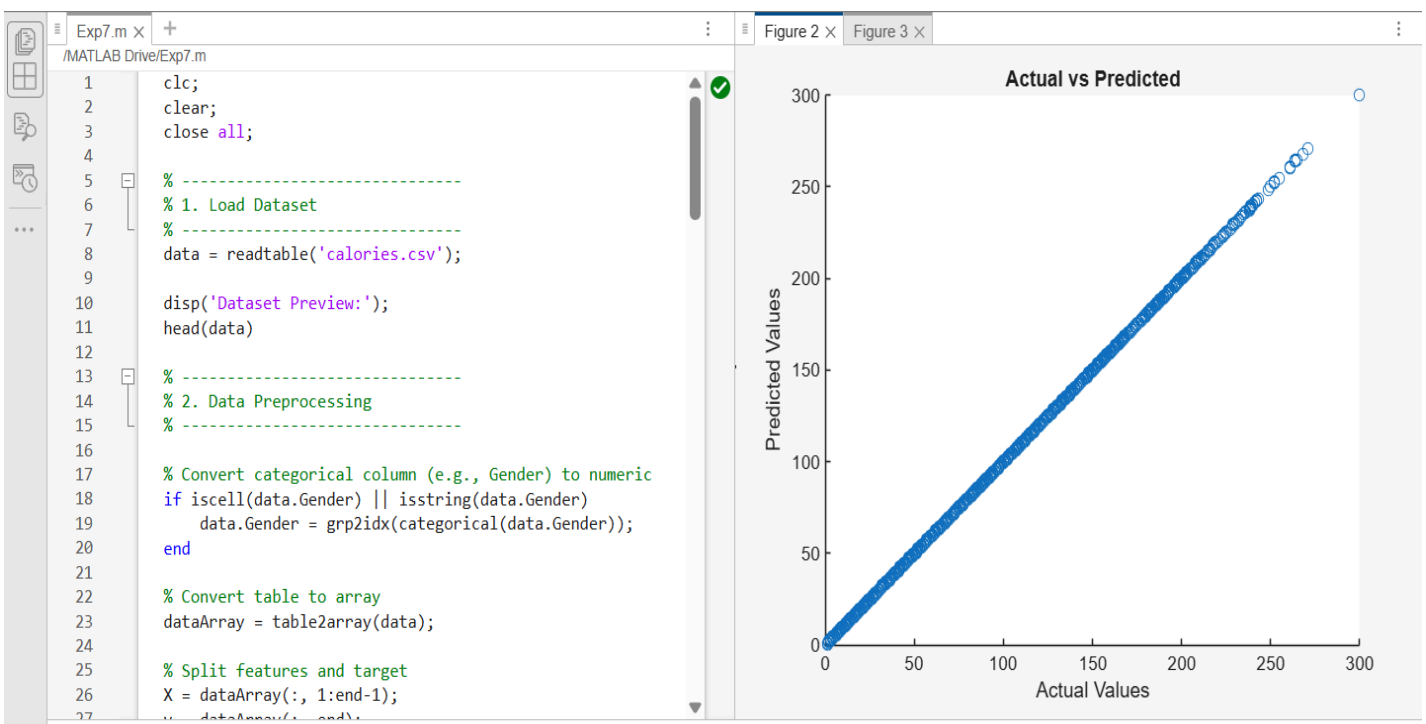









Neural Network Training Regression (plotregression), Epoch 40 , Training finished: ...
File Edit View Insert Tools Help
Training: R=0.99999
Validation: R=0.99999
Test: R=0.99999
All: R=0.99999
Data
Fit
Y = T
Output ~= 1*Target + 0.0078
Output ~= 1*Target + -0.0024
Output ~= 1*Target + -0.003
Output ~= 1*Target + 0.0046
Target


Exp7.m
/MATLAB Drive/Exp7.m
clc;
clear;
close all;
% ----------
% 1. Load Dataset
% ----------
data = readtable('calories.csv');
disp('Dataset Preview:');
head(data)
% ----------
% 2. Data Preprocessing
% ----------
% Convert categorical column (e.g., Gender) to numeric
if iscell(data.Gender) || isstring(data.Gender)
data.Gender = grp2idx(categorical(data.Gender));
end
% Convert table to array
dataArray = table2array(data);
% Split features and target
X = dataArray(:, 1:end-1);
Figure 2
Figure 3
Actual vs Predicted
Predicted Values
Actual Values

Exp7.m

/MATLAB Drive/Exp7.m

```
clc;
clear;
close all;

% -------------------------------
% 1. Load Dataset
% -------------------------------
data = readtable('calories.csv');

disp('Dataset Preview:');
head(data)

% -------------------------------
% 2. Data Preprocessing
% -------------------------------

% Convert categorical column (e.g., Gender) to numeric
if iscell(data.Gender) || isstring(data.Gender)
    data.Gender = grp2idx(categorical(data.Gender));
end

% Convert table to array
dataArray = table2array(data);

% Split features and target
X = dataArray(:, 1:end-1);
```

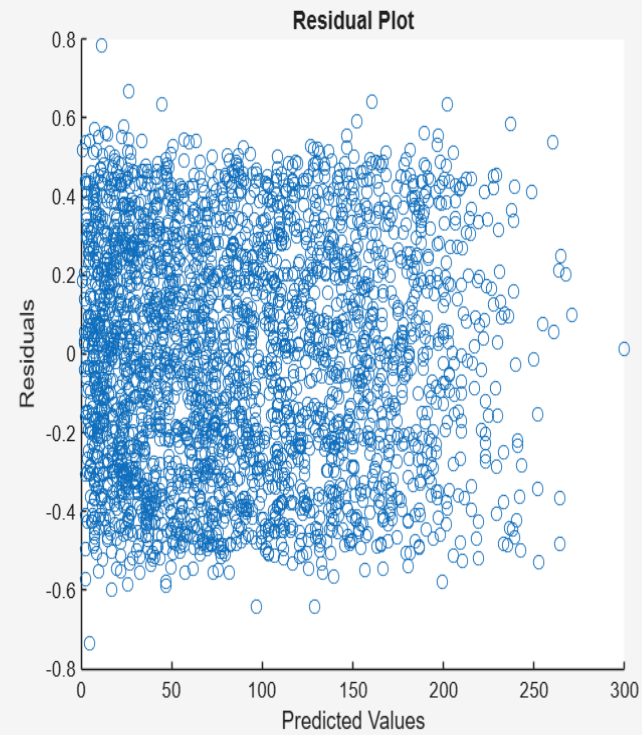



Figure 2 Figure 3